UNIVERSIDADE FEDERAL DE SERGIPE

ATIVIDADE 3 - SISTEMAS OPERACIONAIS (2026.2)

Bruno Amancio Ferreira
Christian Will Silva Santos Nunes
Iasmin Victoria Teixeira Barreto
Marcos Vinícius Dantas Aguiar
Pedro César Figueiredo Carneiro
Sibele Oliveira Cruz
Silas Santos da Silva

VIRTUALIZAÇÃO, NAMESPACES E CGROUPS: LABORATÓRIO DE CONTÊINERES LINUX

SERGIPE
2026

Bruno Amancio Ferreira
Christian Will Silva Santos Nunes
Iasmin Victoria Teixeira Barreto
Marcos Vinícius Dantas Aguiar
Pedro César Figueiredo Carneiro
Sibele Oliveira Cruz
Silas Santos da Silva

VIRTUALIZAÇÃO, NAMESPACES E CGROUPS: LABORATÓRIO DE CONTÊINERES LINUX

Relatório técnico apresentado à Universidade Federal de Sergipe como parte da Atividade 3 da disciplina Sistemas Operacionais, período 2026.2.

SERGIPE
2026

RESUMO

Este relatório apresenta um laboratório reprodutível de contêineres Linux para observar namespaces e limites de cgroup v2. Scripts próprios, sem o kit de apoio do docente, executaram dois contêineres e uma carga aritmética determinística de uma thread. Foram comparadas quotas de 0,5 e 1 CPU, com três repetições por condição, trabalho fixo de 20.000.000 de iterações, memória de 256 MiB, swap adicional zero e limite de 64 tarefas. Todas as execuções concluíram o trabalho com checksum 2589261568 e limites confirmados nos arquivos do kernel. A mediana do tempo decorrido foi 7,0739 s em 0,5 CPU e 3,3840 s em 1 CPU, razão observada de 2,090 vezes. Os contadores registraram throttling nas três execuções de 0,5 CPU. Os contêineres apresentaram handles distintos de PID, mount e network, embora ambos tivessem PID 1 interno. Em montagem somente leitura, a leitura funcionou e uma tentativa controlada de escrita retornou EROFS. Os resultados distinguem isolamento de visão, controle de recursos e restrições de acesso no kernel compartilhado. As limitações incluem três repetições, interferência não isolada do host e ausência de confirmação dos limites ancestrais externos. Não houve inferência de LLM, uso de GPU ou esgotamento deliberado de recursos.

Palavras-chave: Sistemas Operacionais. Contêineres. Namespaces. Cgroups. Virtualização.

SUMÁRIO

1 Contribuições, introdução e problema **3**

2 Ambiente experimental e virtualização **4**

3 Namespaces, montagem e menor privilégio **5**
3.1 Leitura permitida e escrita negada 5

4 Configurações, metodologia e métricas **6**
4.1 Trabalho fixo e correção 6
4.2 Protocolo e escopos de medição 6

5 Resultados de CPU e trabalho por segundo **7**

6 Throttling, memória e interpretação **8**

7 Reprodutibilidade, validação e uso de IA **9**
7.1 Uso crítico e declarado de IA 9

8 Limitações, conclusão e referências **10**

1 CONTRIBUIÇÕES, INTRODUÇÃO E PROBLEMA

Tabela 1: Participação na apresentação e revisão temática

Integrante	Slides e temas da exposição
Bruno Amancio Ferreira	1-2: Abertura, objetivo, organização e percurso do laboratório.
Iasmin Victoria Teixeira Barreto	3-4: Ambiente Windows/WSL2/Docker, diagrama e protocolo experimental.
Pedro César Figueiredo Carneiro	5-6: Namespaces PID, mount e network; numeração dos processos.
Christian Will Silva Santos Nunes	7-8: Carga determinística, checksum, auditoria e demonstração.
Marcos Vinícius Dantas Aguiar	9-10: Limites efetivos do cgroup v2, quotas e throttling.
Sibele Oliveira Cruz	11-12: Seis medições, medianas, gráficos e escopos de memória.
Silas Santos da Silva	13-14: Montagem read-only, menor privilégio, limitações e conclusão.

Os nomes são os da equipe da Atividade 2. A Tabela 1 organiza a participação na exposição e a revisão temática. Cada integrante apresenta dois slides consecutivos, com um único apresentador por slide. A integração e a conferência final dos materiais cabem à equipe. O detalhamento está em `atividade3/docs/contribuicoes.md`; a assistência de IA está declarada na Seção 7.1.

A pergunta norteadora é: *como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente?* O objetivo é responder com comandos, dados e interpretação, conforme o enunciado [5].

A Atividade 2 forneceu experiência com Linux, coleta, dados sintéticos e análise. Seu pipeline utilizava `sleep` para simular geração. Nesta atividade, uma carga aritmética de trabalho fixo permite observar quotas de CPU. Os scripts são próprios e não utilizam o kit de apoio do docente.

Repositório: https://github.com/4rwill/SO_UFS_2026_2_Bruno_Amancio_Christian_Will.
Lote principal: `20261108T013949Z-19921f7f`.
Vídeo: https://drive.google.com/drive/folders/1yPalBhHtxjZmD2yZngRQVawkAElJC-Hj?usp=sharing
Commit de referência dos materiais: `a8347f78e9da`

2 AMBIENTE EXPERIMENTAL E VIRTUALIZAÇÃO

A coleta principal ocorreu em 07/11/2026, aproximadamente entre 22h39 e 22h40, no fuso America/Sao_Paulo. Os registros usam UTC: 08/11/2026, entre 01h39 e 01h40. O inventário distingue o host físico, o ambiente Linux efetivo do daemon e o espaço de usuário da imagem.

Tabela 2: Ambiente efetivamente observado

Camada	Observação
Hardware	Intel Core i5-11260H; 6 núcleos, 12 CPUs lógicas; RAM física reportada: 8.358.518.784 bytes.
Host	Windows 10 Home Single Language, versão 10.0.19045, build 19045.
Linux do daemon	Docker Desktop sobre infraestrutura WSL2; kernel `6.18.54.1-microsoft-standard-WSL2`.
Recursos do daemon	12 CPUs lógicas; 3.989.606.400 bytes de RAM, aproximadamente 3,72 GiB.
Runtime	Docker Desktop 4.96.3; Docker Engine 29.9.2; Compose v5.6.1.
Cgroups	v2, driver cgroupfs; controladores cpu, memory e pids observáveis nos contêineres.
Imagem	Python 3.12 slim; Debian GNU/Linux 13 (trixie); arquitetura amd64.

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

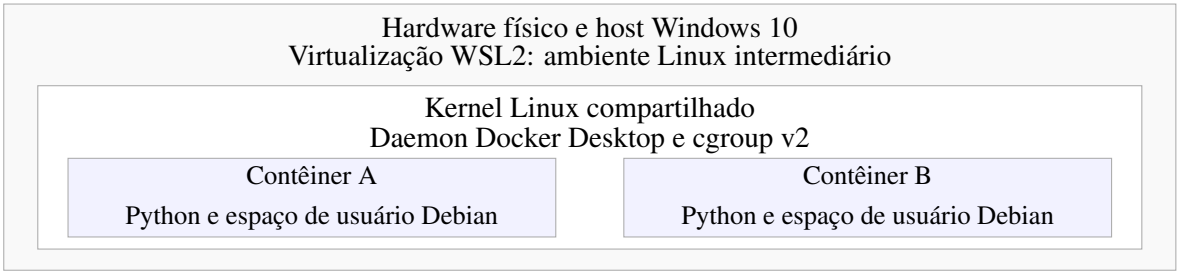


Figura 1: Camadas do ambiente e kernel compartilhado pelos contêineres.

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

Os contêineres compartilham o kernel Linux intermediário; as imagens fornecem bibliotecas e espaço de usuário. O WSL2 estabelece uma camada Linux sobre o Windows. Diferentemente de uma VM com kernel convidado, o contêiner reúne processos sob o kernel compartilhado [1, 3]. Nenhuma VM adicional foi instalada.

A distribuição Ubuntu foi usada na análise e nos testes. O daemon identifica seu ambiente como Docker Desktop; a imagem identifica Debian. `uname` observa o kernel; `/etc/os-release` na imagem descreve seu espaço de usuário.

A imagem oficial foi fixada para todas as observações pelo digest:
`python@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54`

Fontes: `docker-environment.json` e `linux-environment.json` no lote principal; inventário complementar em `atividade3/results/evidence/host-20261108T014504Z/environment.json`.

3 NAMESPACES, MONTAGEM E MENOR PRIVILÉGIO

Dois contêineres independentes da mesma imagem permaneceram ativos durante a coleta. O processo inicial de ambos era Python. Os registros contêm o status de `/proc/1`, os handles e campos selecionados de `inspect`.

Tabela 3: Comparação dos processos e dos três namespaces

Recurso	Contêiner A	Contêiner B
PID no host do daemon	567	612
PID interno	1	1
PID namespace	`pid:[4026532292]`	`pid:[4026532483]`
Mount namespace	`mnt:[4026532287]`	`mnt:[4026532480]`
Network namespace	`net:[4026532405]`	`net:[4026532485]`

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

Os handles diferentes sustentam namespaces distintos [1]. O PID 1 interno é relativo a cada namespace; os PIDs 567 e 612, informados pelo Docker, identificam processos distintos no host do daemon. Isso não demonstra isolamento absoluto nem independência de kernel. A rede foi desativada; não houve teste de tráfego.

Nomes: `so-atv3-8b23fdc285-ns-a` e `so-atv3-8b23fdc285-ns-b`. Handles e PIDs podem mudar em uma reprodução.

3.1 LEITURA PERMITIDA E ESCRITA NEGADA

O diretório sintético foi montado em `/dados` como somente leitura. A aplicação leu `texto.txt` e tentou criar exclusivamente `/dados/tentativa-escrita.txt`. A Tabela 4 apresenta a evidência real.

Tabela 4: Montagem somente leitura e controles de privilégio

Verificação	Observação
Leitura	Documento sintético recuperado integralmente.
Escrita	Negada; errno 30, Read-only file system (EROFS).
Mount observado	Opção `ro` no ponto de montagem `/dados`.
Identidade	UID 65534, sem root.
Capabilities efetivas	`CapEff = 0000000000000000`.
Novos privilégios	`NoNewPrivs = 1`.
Seccomp	Modo 2 observado.

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

Mount namespace define a visão das montagens; a opção `ro` restringe escrita naquele mount; UID e permissões determinam identidade e acesso; capabilities e no-new-privileges reduzem privilégios [1, 4]. O erro EROFS e a opção `ro` sustentam a restrição específica da montagem. Não foram montados home, raiz do host, Docker socket ou diretórios sigilosos.

4 CONFIGURAÇÕES, METODOLOGIA E MÉTRICAS

Um piloto separado de 1.000.000 de iterações levou 0,3095 s sob 0,5 CPU. A calibração escolheu e congelou $N = 20.000.000$ antes das seis execuções, respeitando o teto adotado pelo roteiro. O piloto está preservado em `pilot-0.json` e não entra nas estatísticas principais.

4.1 TRABALHO FIXO E CORREÇÃO

Cada iteração acumula $i^2 + 3i + 7$, módulo 2^{32}, com $0 \leq i < N$. O checksum esperado é calculado independentemente por fórmula fechada:

$$C(N) = \left[\frac{N(N-1)(2N-1)}{6} + \frac{3N(N-1)}{2} + 7N \right] \bmod 2^{32}. \tag{1}$$

Para $N = 20.000.000$, o resultado é 2589261568. As seis execuções concluíram todo o trabalho e concordaram com esse oráculo. A carga utiliza uma thread e memória constante, sem LLM, GPU ou espera simulada.

Tabela 5: Limites efetivos observados antes e depois das cargas

Condição	cpu.max	memory.max	swap.max	pids.max
C1, 0,5 CPU	50000 100000	268435456	0	64
C2, 1 CPU	100000 100000	268435456	0	64

Fonte: snapshots dos cgroups. "swap.max" abrevia memory.swap.max; limites de memória em bytes.

Os valores de `cpu.max` representam tempo agregado por período, em microssegundos. C1 pode consumir 50 ms por período de 100 ms. A quota não fixa afinidade nem reserva um núcleo [2, 3]. A combinação Docker `memory=256m` e `memory-swap=256m` corresponde a 256 MiB totais e swap adicional zero, confirmado por `memory.swap.max`. O limite de 64 tarefas inclui threads e foi confirmado sem provocar esgotamento.

4.2 PROTOCOLO E ESCOPOS DE MEDIÇÃO

Foram usados contêineres novos na ordem C1/C2, C1/C2, C1/C2. Imagem, trabalho e demais limites permaneceram iguais. Todos usaram usuário 65534, capabilities removidas, no-new-privileges, rede desativada e raiz somente leitura. Há watchdog interno de 60 s e externo de 90 s para a carga.

O tempo decorrido mede apenas o laço, excluindo criação do contêiner, inicialização do Python e coleta posterior. Tempo de CPU usa `process_time`. Trabalho por segundo corresponde a iterações/s, sem equivaler a tokens/s. Os deltas de `cpu.stat` abrangem snapshots imediatamente antes e depois do laço e pequena sobrecarga de leitura. A análise compara mediana e variação mínimo-máximo de três observações por condição, sem inferência estatística.

5 RESULTADOS DE CPU E TRABALHO POR SEGUNDO

As seis execuções retornaram status `completed`, exit code zero e correção verdadeira. O checksum comum foi 2589261568. Os registros completos, o piloto e todos os comandos permanecem no lote `20261108T013949Z-19921f7f`.

Tabela 6: Seis execuções principais em ordem alternada

Execução	Wall (s)	CPU do processo (s)	Iterações/s
C1-1	7,0739	3,5227	2.827.285
C2-1	3,4952	3,4947	5.722.124
C1-2	8,2050	4,1136	2.437.526
C2-2	3,3840	3,3829	5.910.140
C1-3	6,3852	3,1909	3.132.258
C2-3	3,2737	3,2732	6.109.225

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

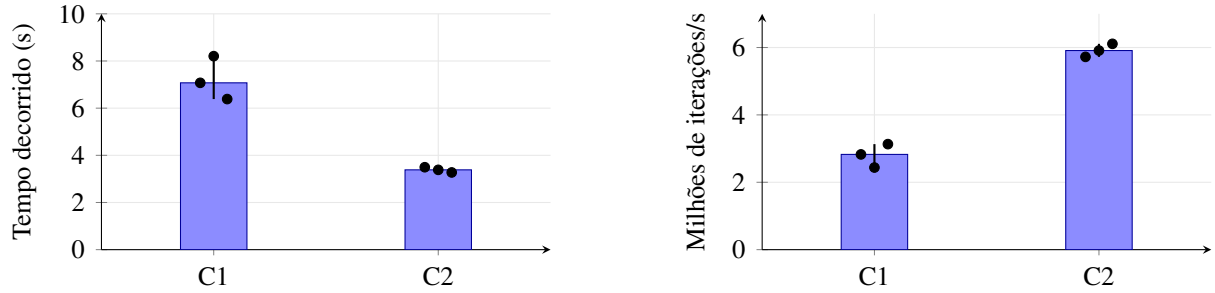


Figura 2: Mesmo trabalho em C1 (0,5 CPU) e C2 (1 CPU). Barras: mediana; traços verticais: mínimo-máximo; pontos: três observações por condição. Piloto excluído.

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

Tabela 7: Medianas e variação de três repetições por condição

Condição	Wall mediano (s)	Mín.–máx. (s)	CPU mediana (s)	Milhões de iterações/s
C1 ($n = 3$)	7,0739	6,3852–8,2050	3,5227	2,8273
C2 ($n = 3$)	3,3840	3,2737–3,4952	3,3829	5,9101

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

A razão entre medianas de wall, C1/C2, foi 2,090. A redução da mediana foi aproximadamente 52,2%. É uma observação deste ambiente, com três repetições, sem demonstrar ganho universal ou exatamente igual a dois. A dispersão de C1 foi maior. Ruído do host, escalonamento e variações de frequência ou temperatura são causas possíveis, sem medição que as isole. A alteração do tempo de CPU entre repetições também recomenda cautela.

6 THROTTLING, MEMÓRIA E INTERPRETAÇÃO

Os limites efetivos, os contadores e os tempos fornecem evidências complementares. Os deltas positivos de `nr_throttled` e `throttled_usec` nas três execuções C1 sustentam a ocorrência de restrição de CPU durante o trabalho [2].

Tabela 8: Deltas de cpu.stat na janela observada

Execução	Usage (s)	nr_throttled	Throttled (s)
C1-1	3,5247	71	3,5448
C2-1	3,4978	0	0
C1-2	4,1167	82	4,0880
C2-2	3,3845	0	0
C1-3	3,1923	64	3,1918
C2-3	3,2748	0	0

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

Em C1, a razão CPU/wall ficou próxima de 0,5; em C2, próxima de 1. Nas três execuções C2, os deltas de throttling foram zero nessa janela. A convergência entre limites, contadores e tempos sustenta a interpretação, sem atribuir toda variação à quota ou concluir ausência de toda restrição externa.

Tabela 9: Escopos distintos das observações de memória

Execução	RSS máximo do processo (MiB)	memory.current final (MiB)	memory.peak do grupo (MiB)
C1-1	16,70	10,89	11,71
C2-1	16,74	11,97	11,97
C1-2	16,77	10,89	10,89
C2-2	16,66	11,93	11,93
C1-3	16,61	10,89	11,95
C2-3	16,78	11,92	11,92

Fonte: elaboração com base no lote experimental da Atividade 3 (2026).

RSS máximo vem de `getrusage` do processo. `memory.current` representa o uso contabilizado no grupo em dois instantes; a Tabela 9 mostra o final. `memory.peak` cobre a vida do cgroup, inclusive a inicialização. Compartilhamento e contabilização diferem; não se exige igualdade entre RSS e memória do grupo. Nenhum desses campos é denominado RAM média [2]. Os bytes exatos e as amostras iniciais estão em `runs.csv`.

O teto de 256 MiB e `pids.max=64` foram confirmados sem OOM deliberado ou criação excessiva de tarefas. O laboratório demonstra configuração e observação dos limites, sem provar todos os comportamentos ao atingi-los. RAM do cgroup não representa quota de VRAM.

7 REPRODUTIBILIDADE, VALIDAÇÃO E USO DE IA

O código está em `atividade3/scripts`. O orquestrador `lab.py` cria snapshots do código e do documento sintético por lote e monta essas cópias read-only. Os contêineres não acessam rede. Identificadores novos preservam resultados anteriores. Comandos, saídas, erros e limites observados ficam registrados.

Commit base da coleta: `9f8ad355769af28f7252a5700905476b6dc6a413`, com a Atividade 3 ainda não commitada. O manifesto inclui os hashes dos scripts e os snapshots preservam o código medido. A organização das duas atividades foi publicada em `7162606`. A versão de referência dos materiais revisados é o commit `a8347f78e9da`.

A auditoria verificou 68 arquivos, imagem fixa, seis execuções, checksums, ordem pelos horários do daemon, limites antes e depois, contadores, privilégios e limpeza. SHA-256 evidencia integridade interna, sem equivaler a assinatura independente da origem das medições. Da raiz do repositório, executar:

```
python atividade3/scripts/audit.py
atividade3/results/raw/20261108T013949Z-19921f7f
```

As duas linhas acima representam um único comando, com o caminho do lote como argumento.

Seis testes passaram no Ubuntu WSL: oráculo aritmético, limites ausentes ou incorretos, leitura do lote e detecção de adulterações, inclusive checksum ou quota alterados com hashes recalculados. As alterações ocorrem somente em fixtures temporárias.

Tentativas auxiliares no Windows falharam por permissões em temporários; seus logs foram preservados. As seis cargas Docker foram válidas. A análise original utilizou Matplotlib 3.10.8; este relatório incorpora os mesmos valores em gráficos vetoriais PGFPlots.

7.1 USO CRÍTICO E DECLARADO DE IA

Ferramenta: Codex. O identificador exato do modelo não estava disponível por fonte verificável nesta sessão e deve ser confirmado pela equipe quando possível. A assistência incluiu implementação, coleta autorizada, auditoria, análise, documentação e adaptação do relatório para LaTeX. Não houve uso ou adaptação do kit do docente.

Prompts relevantes: (1) analisar a viabilidade da Atividade 3 com base na Atividade 2; (2) iniciar a implementação sem o kit de apoio; (3) produzir o relatório em LaTeX no formato do relatório da Atividade 2.

Sugestão adotada: carga determinística com fórmula independente, verificada pelo checksum comum e pelos testes. Sugestão corrigida: não tratar o `/proc` acessível via WSL como equivalente ao host do daemon, após a leitura do PID falhar. Sugestão rejeitada na definição do recorte: utilizar diretamente os tempos do pipeline com `sleep` como benchmark de CPU. As decisões são sustentadas pelo código, logs e metodologia. A IA não substitui revisão, compreensão e responsabilidade acadêmica da equipe.

8 LIMITAÇÕES, CONCLUSÃO E REFERÊNCIAS

A principal ameaça à validade é a ausência de exclusividade e controle do host: há apenas três repetições, alternância fixa e nenhuma medição contínua de carga externa, frequência ou temperatura. A calibração pequena pode sofrer efeitos do período da quota e da inicialização, embora o trabalho principal permaneça fixo e termine com margem para os watchdogs.

Os cgroup namespaces ocultam ancestrais. A tentativa complementar de localizar o PID do daemon no `/proc` acessível via WSL falhou e foi preservada. Limites ancestrais externos não foram confirmados; RAM e CPUs anunciadas pelo daemon descrevem recursos do ambiente intermediário. A limitação impede afirmar ausência de interferência superior, sem invalidar as leituras locais dos limites.

Em um servidor de inferência compartilhado, quotas de CPU limitam a capacidade de uma carga monopolizar tempo de processamento. Tetos de memória e de tarefas reduzem o impacto de alocações e criação excessiva de processos ou threads. Corpus read-only, usuário não root, capabilities removidas e no-new-privileges restringem ações disponíveis. Permanecem riscos associados ao kernel, runtime, host e permissões concedidas; atualizações e configuração adequada continuam necessárias [1, 2, 3].

Conclui-se que o laboratório evidenciou visões distintas de PID, mount e network, limites efetivos de cgroup v2 e negação de escrita no mount read-only. Com trabalho fixo e checksum comum, a mediana de wall caiu de 7,0739 para 3,3840 s ao elevar a quota de 0,5 para 1 CPU. Os contadores sustentam a ocorrência de throttling em C1. Os resultados se restringem a esta carga aritmética, sem avaliar LLM, GPU, rede, armazenamento sob carga, esgotamento de RAM ou isolamento absoluto entre usuários.

A entrega está prevista para 10/11/2026 e a apresentação para 11/11/2026, com 10–12 minutos de exposição e 3–5 minutos de perguntas. O vídeo está indicado na introdução e em `atividade3/VIDEO.md`. A publicação dos materiais no repositório não comprova a submissão no Classroom.

[1] LINUX MAN-PAGES. **namespaces(7)**. https://man7.org/linux/man-pages/man7/namespaces.7.html. Consulta: 7 nov. 2026.

[2] THE LINUX KERNEL DOCUMENTATION. **Control Group v2**. https://docs.kernel.org/admin-guide/cgroup-v2.html. Consulta: 7 nov. 2026.

[3] DOCKER DOCUMENTATION. **Resource constraints**. https://docs.docker.com/engine/containers/resource_constraints/. Consulta: 7 nov. 2026.

[4] DOCKER DOCUMENTATION. **Running containers**. https://docs.docker.com/engine/containers/run/. Consulta: 7 nov. 2026.

[5] UNIVERSIDADE FEDERAL DE SERGIPE. **Sistemas Operacionais: Atividade 3 - Virtualização, contêineres, namespaces e cgroups**. Enunciado revisado, versão de 5 nov. 2026, disponibilizado pela equipe.

[6] EQUIPE GENRESEARCH. **Código, configurações e resultados das Atividades 2 e 3**. 2026. https://github.com/4rwill/SO_UFS_2026_2_Bruno_Amancio_Christian_Will. Lote da Atividade 3: `20261108T013949Z-19921f7f`. A Atividade 2 é referência de organização e formatação; os dados deste relatório provêm exclusivamente do novo lote.